

960121

WRAP UP

Session 10



THE ARCHITECTURAL BEST PRACTICE

Session 1-2: Project Initialization & Data Fetching

- **Version Control (Git):** a clean commit history using Conventional Commits (e.g., `feat:`, `fix:`, `chore:`). This shows professional code management rather than random "updates."
- **UML-First Development:** Before coding, should have a Activity or Sequence Diagram mapping the "round trip" of a GET request.
- **Data vs. UI:** Avoiding "hard-coding." The UI must be dynamically rendered from a data source (JSON/API), separating the *content* from the *container*.

THE ARCHITECTURAL BEST PRACTICE

Session 3: Interactive UI & Event-Driven Architecture

- **Event Delegation:** Instead of many listeners, use one listener on a parent container to manage child interactions (e.g., "Add to Cart" buttons). This proves an understanding of **Memory Efficiency**.
- **Debouncing:** Implementing a delay for search inputs. This is a "Critical Thinking" fix to prevent unnecessary server load and UI flickering.
- **EDA Mindset:** The UI should "react" to events rather than running in a rigid, procedural line.

THE ARCHITECTURAL BEST PRACTICE

Session 4: State Management & Continuity

- **Single Source of Truth:** Managing a global `cartState` array. The UI should only update *after* the state changes.
- **Persistence (localStorage):** Using **Serialization** (`JSON.stringify/parse`) to ensure the user's data survives a page refresh.
- **Hydration Logic:** The application must check for existing data on startup—this is the bridge of "Continuity."

THE ARCHITECTURAL BEST PRACTICE

Session 5-6: Backend & The Architecture of Trust

- **RESTful Design:** Using resource-based URLs (e.g., `/api/products`) and correct HTTP methods (GET, POST).
- **The Gatekeeper Pattern:** Never trusting the client. The backend must re-validate prices and data integrity before processing.
- **Auth Logic:** Moving from plain-text passwords to **Salted Hashing** (`bcrypt`) and Stateless Identity (JWT). This shifts "Identity" from a variable to a secure Token.

THE ARCHITECTURAL BEST PRACTICE

Session 7: The Transactional Checkout

- **Body Parsing:** Proper use of "unpacking" middleware (`express.json()`) to handle complex POST payloads.
- **Server-Side Validation:** Using **RegEx** or validation libraries to ensure the data is "Safe" (Sanitization) and "Sensible" (Validation).
- **Atomic Transactions:** Ensuring that if a checkout step fails, the system doesn't leave the user in a "broken" state (e.g., charged but no order).

THE ARCHITECTURAL BEST PRACTICE

Session 8: Relational Persistence

- **Data Normalization:** Moving away from duplicated JSON data into a **Relational Schema** (Users, Products, Orders).
- **Referential Integrity:** Using **Primary Keys** and **Foreign Keys** to link data, ensuring you can't have an order for a non-existent user.
- **SQL Security:** Mandatory use of **Parameterized Queries** (? placeholders) to prevent SQL Injection.

THE ARCHITECTURAL BEST PRACTICE

Session 9: Modularization & Scaling

- **Controller-Route-Service Pattern:** A strict "Separation of Concerns." The **Route** defines the entry, the Controller manages the flow, and the **Service** handles the heavy business logic.
- **Fault Tolerance:** Designing the app so that a failure in one logical module doesn't necessarily crash the entire system.
- **Microservice Thinking:** Identifying where "Logical Boundaries" could become "Physical Boundaries" (moving a module to its own server).

THE ARCHITECTURAL BEST PRACTICE

Session 10: The "Go-Live" Audit

- **Environment Security:** Absolute zero tolerance for hard-coded secrets. All sensitive keys must be in `.env` and excluded from Git via `.gitignore`.
- **Portability:** The project must be "Zero-Config"—able to run on any machine using only `npm install` and `npm start`.
- **Graceful Error Handling:** Hiding stack traces from the user while logging them for the developer.

960121

MINI PROJECT ASSIGNMENT

Session 10



THE SMART-NICHE MARKETPLACE

- **Group Size:** 3 Students
- **Duration:** Final 4–5 Weeks
- **Core Concept:** Students choose a specific niche (e.g., *Sustainable Fashion*, *Limited Edition Sneakers*, or *Fresh Grocery*) where the business logic is more complex than a standard shop.

THE SMART-NICHE MARKETPLACE

1. The "Resell" Product Exchange (Verification Logic)

- **The Niche:** A high-end marketplace for limited-edition products where prices fluctuate based on demand, such as sneakers like Nike and Supreme or comic figures/books like Marvel themed
- **The Architectural Twist: Multi-Step Status Logic.**
 - **The Logic:** An order isn't just "Done" once paid. It moves through states: Pending → Received at Warehouse → Authenticated → Shipped.
 - **Challenge:** You must design a database table that tracks the "History" of an order status and prevents a user from clicking "Received" if the status is still "Pending."

THE SMART-NICHE MARKETPLACE

2. The "Community Garden" Share (Geographic/Time Logic)

- **The Niche:** Neighbors selling or swapping surplus home-grown produce (tomatoes, honey, eggs).
- **The Architectural Twist: Expiration & Pickup Windows.**
 - **The Logic:** Fresh produce has a "Best Before" date stored in the database.
 - **Challenge:** The Catalog Service must filter out items automatically if `currentDate > expiryDate`. Additionally, the checkout must require the user to select a specific "Pickup Window" from a pre-defined list of available slots.

THE SMART-NICHE MARKETPLACE

3. The "Vintage Vinyl" Crate (Condition-Based Pricing)

- **The Niche:** Rare records where one album might have five different listings based on its physical condition (Mint, Good, Poor).
- **The Architectural Twist: One-to-Many Inventory mapping.**
 - **The Logic:** Instead of one "Product" having one "Price," the `Product` table links to an `Inventory` table where each row has a `condition_grade` and a `unique price`.
 - **Challenge:** The Frontend must show a dropdown for "Condition," and the Backend must ensure the `OrderService` pulls the specific price tied to that specific grade.

THE SMART-NICHE MARKETPLACE

4. The "Skill-Share" Workshop (Capacity & Seat Logic)

- **The Niche:** Booking seats for live online classes (e.g., "Intro to AI," "Sourdough Bread Making").
- **The Architectural Twist: Real-time Seat Reservation.**
 - **The Logic:** Every "Product" (Class) has a `max_capacity`.
 - **Challenge:** During the Checkout POST, the server must check `current_bookings < max_capacity`. If it's full, the "Gatekeeper" must reject the payment and return a 409 Conflict status code.

THE SMART-NICHE MARKETPLACE

5. The "Eco-Refill" Subscription (Frequency Logic)

- **The Niche:** Sustainable household cleaning products that offer users a discount when they subscribe to monthly refills.
- **The Architectural Twist: Conditional Subscription Discounts.**
 - **The Logic:** A user can buy a one-time bottle for \$15 or a "Subscription" for \$12.
 - **Challenge:** The `OrderService` must record whether the order is a `one-time` or `recurring` type. If it is recurring, the logic must automatically apply a 20% discount to the line item, but only if the user is **Logged In** (Session 6 integration).

BASE REQUIREMENTS (THE "MUST-HAVES")

You must use all the knowledge in “The Architectural Best Practice” to implement:

- Dynamic product catalog display,
- Interactive products filtering (keyword, price, category),
- Registration and Login,
- Add product to cart or Booking
- Check out or Placing an order

“Bypass the Payment Process”

1-3 SCORING SYSTEM

Category	Best Practice Point
1. Version Control	Conventional Commits & Git Flow
2. Data Flow	Separation of Content & UI
3. Interaction	Event Delegation & Debouncing
4. State	Single Source of Truth & Continuity
5. Security (Auth)	The Architecture of Trust
6. Security (API)	The Gatekeeper Pattern
7. Persistence	Relational Integrity (SQL)
8. SQL Safety	Parameterized Queries
9. Structure	Controller-Route-Service (SoC)
10. Deployment	The "Zero-Config" & .env Audit

1-3 SCORING SYSTEM

How to Calculate the Final Grade

- **Score 3 (Mastery):** Student can explain why they chose a pattern. Code is modular, secure, and documented.
- **Score 2 (Competency):** The app works, but some best practices are missing.
- **Score 1 (Basic):** Implemented with very little architectural logic.
- **Score 0 (404 Not Found):** Best Practice NOT implemented

BONUS POINTS

To earn extra points, groups must choose **one** of these "Architectural Challenges."

A. (3 points) The "Stock-Check" Store: Concurrency Logic

- **The Business Problem:** High-demand items (like sneaker drops). Two people click "Buy" on the last item at the same time.
- **The Architectural Task:** Implement a **Pre-Checkout Inventory Check**.
- **Logic:** The `OrderService` must query the `Product` table and verify `stock > 0` within the transaction block before allowing the POST to succeed. If stock is 0, return a specific "Out of Stock" error.

BONUS POINTS

B. (3 points) The "Dynamic Discount" Engine: Calculation Logic

- **The Business Problem:** Marketing wants to give 10% off if the cart total > \$200, OR 15% off if they buy more than 3 items from the "Fresh" category.
- **The Architectural Task:** Create a `DiscountService`.
- **Logic:** This service must intercept the `Checkout POST` payload, run the math on the server (Gatekeeper Pattern), and return a "Re-calculated Total." You must prove the calculation happened on the **Backend**, not the Frontend.

BONUS POINTS

C. (5 points) Personalized Recommendations: Relational Logic

- **The Business Problem:** "Users who bought X also liked Y."
- **The Architectural Task:** Complex **SQL Joins**.
- **Logic:** Write a SQL query that finds the most common products appearing in orders placed by *other users* who purchased the item currently being viewed.

GROUP MEMBER ROLES

- **Encouraging Collaboration:**
 - **Lead Architect (Backend/DevOps):** Responsible for the `Service` layer, SQL Schema, `.env` security, and the "Go-Live" Audit.
 - **Integration Engineer (API/State):** Responsible for the "Request/Response Map," `fetch()` logic, JWT handling, and `localStorage` continuity.
 - **UX Engineer (Frontend/Interaction):** Responsible for the interactive UI, Event Delegation, Debouncing, and the CSS/Layout logic.

PEER ASSESSMENT

Rate your teammates on a scale of **1–5** for each category.

Category	Deception	Weight
1. Logic Contribution	Did they contribute to the business and programming logic design or the SQL Schema?	25%
2. Technical Reliability	Did their code follow the "Gatekeeper" and "Security" patterns we learned?	25%
3. Git & Integration	Did they commit regularly? Did they help resolve merge conflicts or integrate modules?	25%
4. Communication	Did they participate in the "Architectural Handwork" (drawing the maps/logic trees)?	25%

PEER ASSESSMENT

Rate your teammates and provide a comment for each category.

Your comments must provide specific examples of their contribution to the code or logic.

<https://cmu.to/960121PeerReview>

